

Why Did These Optimizers Become Famous?

Vatsav Reddy

Under the Guidance of
Prof. Praveen Parchuri

International Institute of Information Technology, Hyderabad

May 21, 2026

Contents

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary

Current Section

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary

Prof. Praveen's Question

The Question

Out of possibly **infinitely many** optimizers one could design mathematically, *why did these specific few become famous?*

The Surface Answer

- **Not** because they have elegant equations
- **Not** because the papers were mathematically beautiful
- **Yes** because each solved a *painful, real-world engineering bottleneck* in deep learning

The Deep Scientific Answer

Almost all famous optimizers are approximating **Newton's method** cheaply under hardware, memory, and stochasticity constraints:

$$w_{t+1} = w_t - H^{-1} \nabla \mathcal{L}(w_t)$$

Storing H costs $O(d^2)$; inverting costs $O(d^3)$ — **impossible** for deep learning.

The core question each optimizer answers is:

"How can we imitate $H^{-1} \nabla \mathcal{L}$ without ever computing H ?"

The Hidden Evolution of Optimizers

Optimization Research as Failure-Mode Fixing

“What specific failure of the previous optimizer prevented neural networks from scaling?”

Every famous optimizer fixed **one dominant failure mode**.

The Evolutionary Chain

Optimizer	Failure It Fixed	Intelligence Added
GD	No optimizer existed	Move downhill
SGD	Full-batch too expensive	Compute at scale
Momentum	Zig-zagging, flat regions	Remember the past
NAG	Blind overshooting	Anticipate the future
AdaGrad	Sparse features under-trained	Adapt per feature
RMSProp	Learning rate dies	Forget old history
Adam	Too much manual tuning	Combine all of the above
AdamW	Regularisation entangled in Adam	Separate opt. from reg.

Current Section

- 1 The Question & The Deep Answer
- 2 **Vanilla GD & SGD: The Computation Bottleneck**
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary

Vanilla GD — Why It Became the Foundation

Update Rule & Hessian Interpretation

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t) = w_t - \underbrace{\eta I}_{\approx H^{-1}} \nabla \mathcal{L}(w_t)$$

GD assumes $H^{-1} \approx \eta I$ — **same curvature in every direction** (isotropic).

Why It Fails on Real Loss Surfaces

Neural network loss surfaces have highly **anisotropic** curvature: $\lambda_{\max} \gg \lambda_{\min}$ (ill-conditioned Hessian).

Result: severe zig-zagging in narrow ravines; tiny gradients on gentle slopes $\Rightarrow$ very slow movement.

Why SGD Became Famous

- Datasets became huge: ImageNet, web-scale NLP, recommendation systems
- Full-batch GD: **one update per epoch** (millions of calculations before any parameter move)
- SGD: **N updates per epoch** — enabled online learning and memory-efficient training
- Made training on millions of samples **feasible** $\Rightarrow$ unlocked the deep learning boom

Real-World Bottleneck Solved

Computational scalability — not a mathematical improvement, a *hardware and data reality* fix.

Current Section

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics**
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary

Momentum — Temporal Curvature Sensing

Update Rule

$$u_t = \beta u_{t-1} + \eta \nabla w_t, \quad w_{t+1} = w_t - u_t$$

Expanding recursively: $u_t = \eta \sum_{i=1}^t \beta^{t-i} \nabla w_i$ — an **exponentially weighted moving average** of past gradients.

Scientific Interpretation

- **Flat valley (low curvature):** gradients stay *aligned* for many steps $\Rightarrow$ EMA *amplifies* — optimizer accelerates
- **Steep direction (high curvature):** gradients *flip sign* rapidly $\Rightarrow$ EMA *cancels oscillations*

Momentum **implicitly senses curvature anisotropy** through temporal averaging — a cheap approximation to second-order dynamics.

Real-World Bottleneck Solved

SGD zig-zagged in ravines; neural network loss surfaces are **not** spherical bowls — they are ravines, curved valleys, anisotropic surfaces. Momentum matched this geometry. *“Gain confidence, take bigger steps in same direction.”*

Nesterov Accelerated Gradient — Predictive Curvature Correction

Momentum: Standing Here

- 1 Stand at w_t
- 2 Check slope *here*
- 3 Move (may overshoot)

NAG: Look Ahead First

- 1 Peek at $w_t - \beta u_{t-1}$
- 2 Check **future** slope
- 3 Adjust *before* moving

Scientific Interpretation: Hessian Appears Explicitly

Taylor-expand the look-ahead gradient:

$$\nabla \mathcal{L}(w_t - \beta u_{t-1}) \approx \nabla \mathcal{L}(w_t) - \beta H u_{t-1}$$

The term $\beta H u_{t-1}$ is a **curvature correction**. NAG is thus a first-order approximation to second-order predictive correction — this is the deep reason it outperforms plain Momentum.

Real-World Bottleneck Solved

Deeper networks became unstable; momentum's blind acceleration caused many U-turns. NAG's lookahead gave **smarter, more stable acceleration** — “*Running fast while looking ahead.*”

Current Section

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity**
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary

AdaGrad — Diagonal Hessian Approximation

Motivation: Sparse Features

$$\nabla w^1 = (f(x) - y) \cdot f(x) \cdot (1 - f(x)) \cdot x^1$$

If x^1 is sparse (mostly 0): $\nabla w^1 \approx 0$ for most inputs $\Rightarrow w^1$ gets very few updates despite possibly being *critical* (e.g. rare words: “earthquake”, “fraud”).

AdaGrad Update & Newton Connection

$$v_t = v_{t-1} + (\nabla w_t)^2, \quad w_{t+1} = w_t - \underbrace{\frac{\eta}{\sqrt{v_t + \varepsilon}}}_{\approx D_t} \nabla w_t \approx w_t - \eta H^{-1} \nabla w_t$$

where $D_t = \text{diag}((\sqrt{v_t} + \varepsilon)^{-1})$ is a **diagonal approximation to H^{-1}** .

For quadratic loss $\mathcal{L} = \frac{1}{2} w^T H w$, large-curvature directions produce consistently large gradients, so v_t estimates curvature magnitude — **diagonal preconditioning from gradient statistics**.

Real-World Bottleneck Solved + Fatal Flaw

Solved sparse feature under-training at Google-scale NLP and ad-ranking. Fatal flaw: $v_t \uparrow \infty \Rightarrow \text{effective } \eta \rightarrow 0$ — AdaGrad gets stuck near convergence.

RMSProp — Adaptive Local Curvature via EMA

Fix: Exponential Moving Average of Squared Gradients

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla w_t)^2 \quad (\beta > 0.9)$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

Now $v_t \approx \mathbb{E}[g^2]$ — the **local second moment** of gradients, not the unbounded cumulative sum.

Fisher Information Connection

For many probabilistic models:

$$\mathbb{E}[g^2] \approx \mathcal{F} \quad (\text{diagonal Fisher Information})$$

The Fisher Information is a **natural curvature measure** in probability space. RMSProp therefore approximates H^{-1} through *local curvature statistics* — **does not kill the learning rate**.

Real-World Bottleneck Solved

Neural networks are **non-stationary**: gradients from 1000 steps ago should not dominate today. EMA decay introduces *forgetting and recency awareness*, matching real training dynamics. Practical success > theoretical elegance.

Current Section

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard**
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary

Adam — Stochastic Diagonal Quasi-Newton Method

Adam = RMSProp + Momentum + Bias Correction

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla w_t && \leftarrow \text{1st moment (direction)} \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla w_t)^2 && \leftarrow \text{2nd moment (curvature)} \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} && \leftarrow \text{bias correction} \\ \beta_1 &= 0.9, \quad \beta_2 = 0.999, \quad \varepsilon = 10^{-8}\end{aligned}$$

$$\boxed{w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \cdot \hat{m}_t} = w_t - \underbrace{\eta \underbrace{D_t^{-1}}_{\approx H^{-1}}}_{\approx H^{-1}} \hat{m}_t$$

Adam is a **stochastic, diagonal, quasi-Newton method** with momentum.

Real-World Bottleneck Solved

Before Adam: careful per-task LR tuning, optimizer instability. After Adam: one set of defaults works across CNNs, RNNs, Transformers, GANs, and diffusion models.

Engineering practicality beats theoretical purity — even though Adam does not converge in some cases, industry adopted it universally because it *reduces human suffering*.

AdamW — Decoupled Geometry and Regularisation

The Hidden Problem in Adam + L2

Standard Adam with L2 regularisation sets $g_t \leftarrow \nabla \mathcal{L}(w_t) + \lambda w_t$. Now λw_t enters the curvature accumulator v_t , **entangling regularisation with adaptive scaling** — weights that are large get a smaller effective learning rate, which is not true weight decay.

AdamW Fix: Apply Weight Decay After the Adaptive Step

$$w_{t+1} = w_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \underbrace{\eta \lambda w_t}_{\text{true weight decay}}$$

- **Optimisation geometry:** $\hat{m}_t / \sqrt{\hat{v}_t}$
- **Regularisation geometry:** λw_t (independent of curvature scaling)

Cleaner decoupling $\Rightarrow$ better generalisation and stability for over-parameterised models.

Real-World Bottleneck Solved

Transformers are hugely over-parameterised and sensitive to regularisation. AdamW became **the standard for LLMs and ViTs** because decoupled weight decay improved generalisation at scale.

Current Section

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply**
- 7 Summary

Grand Unification: Every Optimizer Approximates H^{-1}

Newton's Ideal (Unachievable in Practice)

$$w_{t+1} = w_t - H^{-1} \nabla \mathcal{L}(w_t) \quad [\text{cost: } O(d^2) \text{ storage, } O(d^3) \text{ inversion}]$$

The $O(d)$ Approximation Hierarchy

Optimizer	H^{-1} Approximation	Key Insight
GD	ηI	Isotropic; ignores curvature
Momentum	EMA filter on gradients	Temporal curvature sensing
NAG	$I - \beta H$ (1st-order)	Curvature correction via lookahead
AdaGrad	$\text{diag}(\sum g^2)^{-1/2}$	Cumulative diagonal preconditioning
RMSProp	$\text{diag}(\mathbb{E}[g^2])^{-1/2}$	Local curvature $\approx$ Fisher info
Adam	EMA diagonal + smoothed gradient	Quasi-Newton with momentum
AdamW	Adam + decoupled λw	Separate geometry & regularisation

Why Only These Survive: The Brutal Filter

Survival Criteria — All Must Be Satisfied

- $O(d)$ **cost**: scales to billions of parameters on GPU hardware
- **Stochastic robustness**: stable under mini-batch gradient noise, no full-batch requirement
- **Minimal tuning**: default hyperparameters transfer across tasks without per-problem search
- **GPU parallelism**: element-wise operations; no sequential or high-memory-bandwidth steps
- **Predictable behaviour**: converges (or diverges predictably) on standard architectures
- **Reduces human effort**: lowers the number of manual engineering decisions during training

The Core Insight

There are *infinitely many* mathematically valid optimizers. Most fail at least one criterion above. The few that satisfy **all criteria simultaneously** are the names you know.

The history of optimizers is the history of cheap approximations to H^{-1} under real-world constraints.

Current Section

- 1 The Question & The Deep Answer
- 2 Vanilla GD & SGD: The Computation Bottleneck
- 3 Momentum & NAG: Curvature-Aware Dynamics
- 4 AdaGrad & RMSProp: Per-Feature Adaptivity
- 5 Adam & AdamW: The Default Standard
- 6 Grand Unification: Approximating Newton Cheaply
- 7 Summary**

Summary

Engineering View

- **GD / SGD**: feasibility at data scale; N updates per epoch vs. 1
- **Momentum**: smooth traversal of anisotropic loss surfaces; EMA velocity
- **NAG**: lookahead curvature correction; reduces U-turns in deep nets
- **AdaGrad**: per-feature adaptive η ; sparse feature parity
- **RMSProp**: non-stationary training dynamics; EMA prevents $\eta \rightarrow 0$
- **Adam**: plug-and-play default; combines momentum + adaptivity + bias correction
- **AdamW**: Transformer regularisation; decoupled weight decay

Scientific View (H^{-1} Approx.)

- **GD**: $H^{-1} \approx \eta I$
- **Momentum**: temporal curvature sensing via gradient EMA
- **NAG**: $\nabla \mathcal{L}(w - \beta u) \Rightarrow Hu$ term appears
- **AdaGrad**: cumulative diagonal H^{-1} estimator
- **RMSProp**: EMA diagonal $\approx$ Fisher information
- **Adam**: stochastic diagonal quasi-Newton
- **AdamW**: Adam with decoupled curvature & regularisation

One-Line Answer for Prof. Praveen

These optimizers became famous **not** because they were mathematically possible, but because each solved a **dominant real-world bottleneck** — computation, instability, sparse features, convergence speed, adaptivity, or generalisation — while remaining $O(d)$ and robust enough to use as a **default**.

Thank You

Questions?